Activation Functions in Neural Networks

Kritanta Saha, Asst. Prof., Dept. of CSE, SNU

March 18, 2026

Activation Functions: Definition & Importance

Definition

- Introduce **non-linearity**
- Transform:

$$z = w^T x + b, \quad a = f(z)$$

- Without activation:
  - Network $\rightarrow$ linear mapping

Importance

- Enables **universal approximation**
- Controls:
  - Gradient propagation
  - Output scaling
  - Optimization stability
- Critical for deep networks

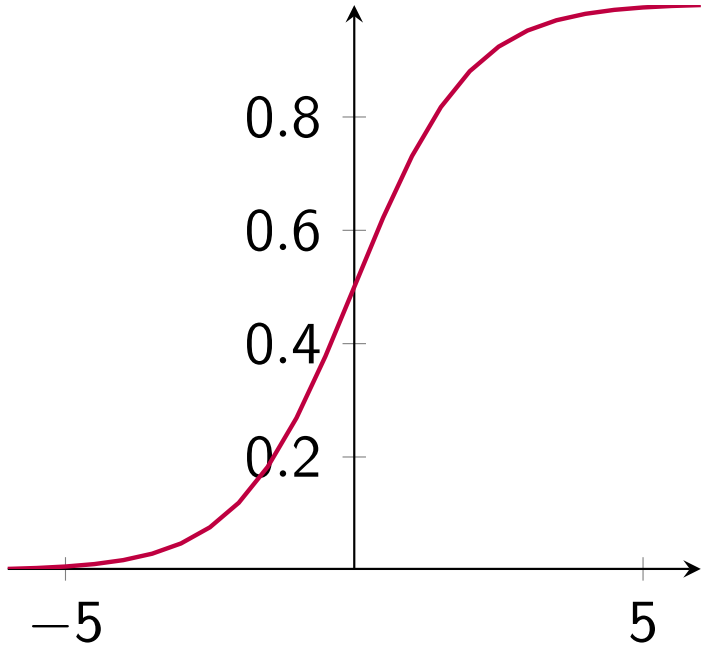
Sigmoid (Logistic Function)

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Derived from logistic regression
- Maps $\mathbb{R} \to (0, 1)$ (probability)
- Saturating at extremes
- Derivative:

$$f'(x) = f(x)(1 - f(x))$$

- Gradient $\to 0$ for large $|x|$



Purpose: Binary classification
Advantages: Smooth, probabilistic
Disadvantages:

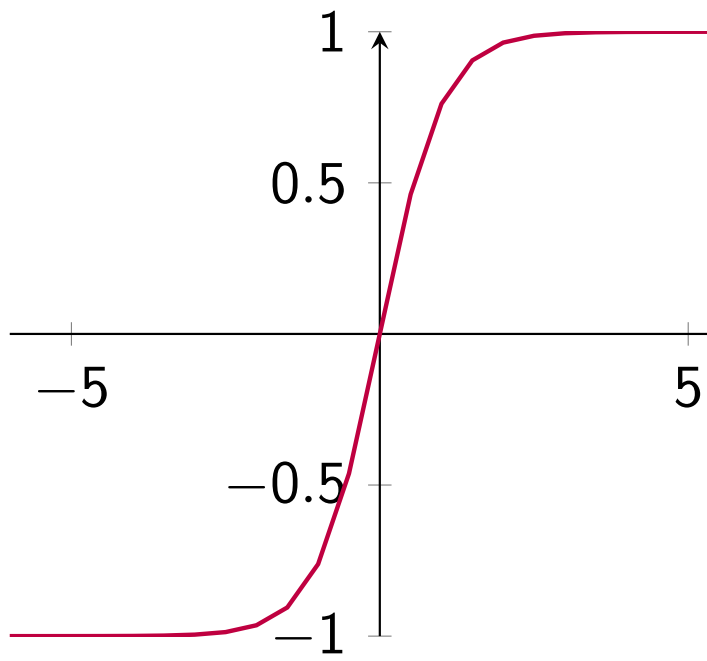
- Vanishing gradient

Tanh (Hyperbolic Tangent)

$$f(x) = \tanh(x)$$

- Scaled sigmoid: $[-1, 1]$
- Zero-centered $\rightarrow$ better optimization
- Derivative:
$$1 - \tanh^2(x)$$
- Still saturates $\rightarrow$ vanishing gradients

Purpose: Hidden layers
Advantages: Zero-centered
Disadvantages: Gradient saturation



ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Piecewise linear, non-saturating ($x > 0$)
- Sparse activation (many zeros)
- Derivative:

$$f'(x) = 1 \text{ or } 0$$

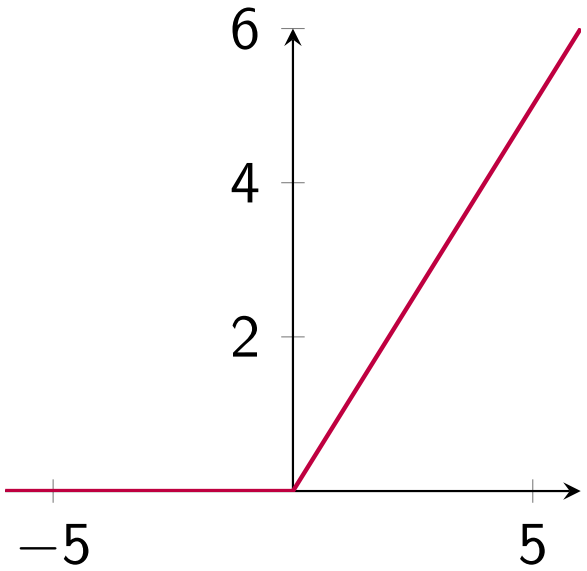
- Enables efficient deep learning

Purpose: Hidden layers

Advantages:

- Fast computation
- Avoids vanishing gradient

Disadvantages: Dying ReLU



Leaky ReLU (Leaky Rectified Linear Unit)

$$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

- Extends ReLU with small negative slope
- Prevents zero gradient region
- Maintains gradient flow

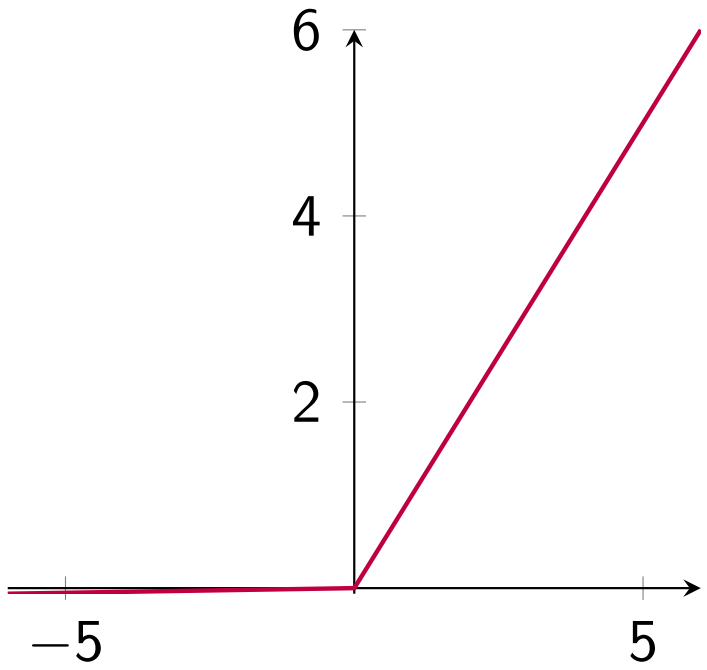
Purpose: Fix dying ReLU

Advantages:

- Non-zero gradient everywhere

Disadvantages:

- Requires hyperparameter α



ELU (Exponential Linear Unit)

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

- Smooth exponential negative region
- Reduces bias shift toward positive mean
- Improves convergence speed

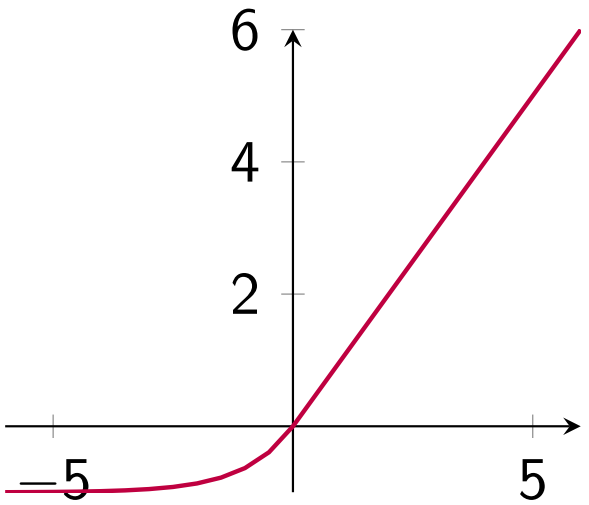
Purpose: Faster training

Advantages:

- Smooth gradients

Disadvantages:

- Computational cost



Swish (Self-Gated Activation)

$$f(x) = x \cdot \sigma(x)$$

- Combines linear and sigmoid
- Self-gating mechanism
- Non-monotonic $\rightarrow$ better representation

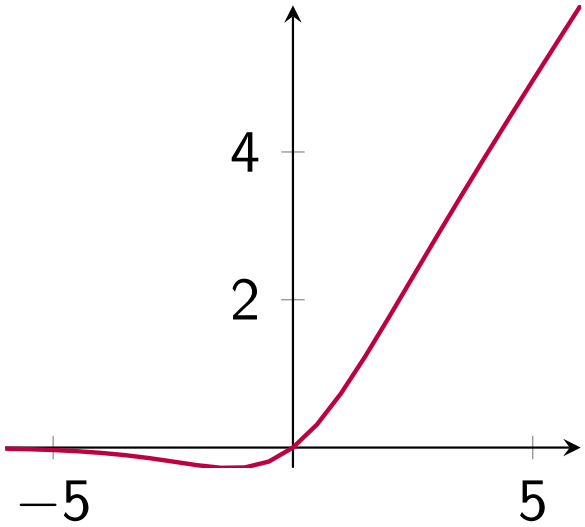
Purpose: Deep networks

Advantages:

- Smooth gradient flow

Disadvantages:

- Computationally expensive



GELU (Gaussian Error Linear Unit)

$$f(x) = x\Phi(x)$$

- Based on Gaussian CDF
- Probabilistic gating of inputs
- Smooth alternative to ReLU

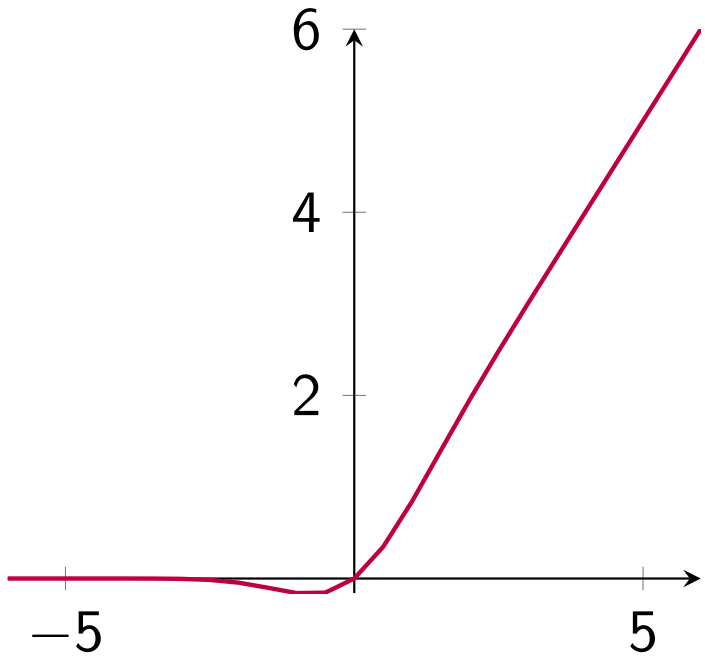
Purpose: Transformers

Advantages:

- Better generalization

Disadvantages:

- High computational cost



Softplus (Smooth ReLU)

$$f(x) = \ln(1 + e^x)$$

- Smooth approximation of ReLU
- Derivative equals sigmoid
- Fully differentiable everywhere

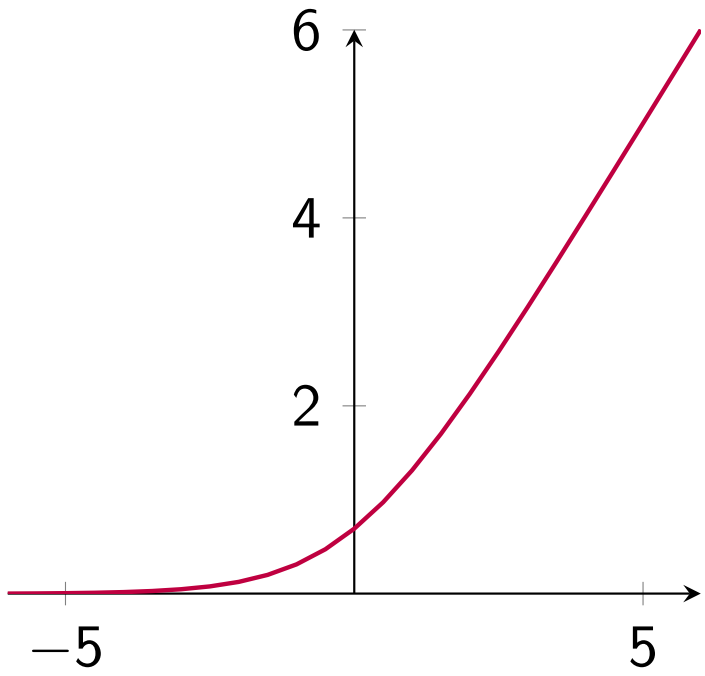
Purpose: Smooth models

Advantages:

- No sharp transitions

Disadvantages:

- Computational cost



Softmax (Normalized Exponential Function)

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

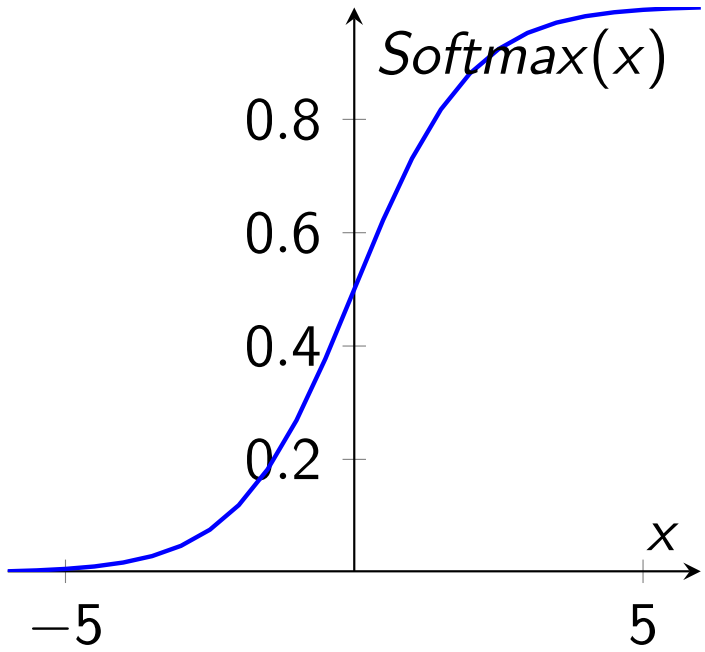
- Converts logits to probability distribution
- Ensures outputs sum to 1
- Sensitive to relative differences

Purpose: Multi-class classification

2-Class Simplification:

$$f(x) = \frac{e^x}{e^x + 1}$$

- Equivalent to **Sigmoid**
- Shows probability transition



Sigmoid vs Softmax

Aspect	Sigmoid (Logistic Function)	Softmax (Normalized Exponential)
Formula	$\frac{1}{1+e^{-x}}$	$\frac{e^{x_i}}{\sum_j e^{x_j}}$
Output Range	(0,1)	(0,1)
Output Sum	Not constrained	Sum $= 1$
Dependency	Independent outputs	Interdependent outputs
Interpretation	Probability per class	Probability distribution
Use Case	Binary / Multi-label	Multi-class classification
Competition	No competition	Classes compete
Typical Layer	Output layer (binary)	Output layer (multi-class)

Key Insight:

- Sigmoid treats each output independently
- Softmax enforces competition among classes

Activation Function Comparison

Func	Range	Zero-Centered	Vanishing Grad	Derivative Behavior	Cost	Use
Sigmoid	(0,1)	No	Yes	Small at extremes	High	Binary output
Tanh	(-1,1)	Yes	Yes	Larger than sigmoid	High	Hidden (older)
ReLU	$[0,\infty)$	No	No	1 or 0 (sparse)	Very Low	Default hidden
Leaky ReLU	$(-\infty,\infty)$	No	No	Small slope for $x < 0$	Very Low	Fix dying ReLU
ELU	$(-\alpha,\infty)$	Approx Yes	Less	Smooth, non-zero	Medium	Faster learning
Swish	$(-\infty,\infty)$	No	No	Smooth, non-monotonic	High	Deep nets
GELU	$(-\infty,\infty)$	No	No	Probabilistic smooth	High	Transformers
Softplus	$(0,\infty)$	No	No	Smooth ReLU-like	High	Smooth alt
Softmax	(0,1)	–	–	Depends on all inputs	High	Multi-class

Insights:

- ReLU is fastest $\rightarrow$ best for large-scale models
- Sigmoid/Tanh suffer from gradient saturation
- GELU/Swish improve smooth gradient flow but are costly

Conclusion

- Activation functions enable non-linear representation
- ReLU is most widely used
- Sigmoid/Softmax for outputs
- GELU/Swish dominate modern deep learning
- Choice impacts convergence and performance